- Zetta exposes Websocket endpoints to stream real-time events. This model of merging Hypermedia with Websocket streaming is acknowledged as Reactive Hypermedia.
- It can support almost all device protocols, and mediate them to HTTP. Connections between servers are also persistent, so that we can use the seamless services between servers in the cloud.
- We can create stateless applications in Zetta servers. Applications can be useful to connect devices, run different queries and interact between them. We can also write queries and triggers so that whenever new devices are attached, we will be notified.

Zetta architecture

The Zetta server: The Zetta server is the main component of Zetta, which contains different sub-components like drivers, scouts, server extensions and apps. A Zetta server will run on a hardware hub such as BeagleBone Black, Raspberry Pi or Intel Edison. The server manages interactions between all the sub-components in order to communicate with devices and generate APIs, by which consumers can interact.

Scouts: Scouts provide a discovery mechanism for devices on the networks or to those which require system resources to understand and communicate with a specific type of protocol.

Scouts help Zetta to search for devices based on a particular protocol. They also fetch specific information about devices and whether those devices have already interacted with Zetta or not. They maintain security credentials and related details while communicating.

Drivers: Drivers are used to represent the devices in a state machine format. They are used for modelling devices and physical interaction between devices. Device models are used to generate different API calls.

Server extensions: These are used for extending functionalities. They are in a pluggable mode, and deal with API management, adding additional securities, etc.

Registry: This is a database for the Zetta server, which stores information about the devices connected to the server. It is a persistence layer.

Secure linking: We can establish secure and encrypted tunnelling between different servers while communicating. This takes care of firewalls and network settings.

Apps: Apps that are used for different interactions between devices or to fetch and process some data are created in JavaScript. Apps can be created based on sensor streams or changes in devices. They can be used to track certain kinds of events that happen in systems.

Zetta deployment

Now let us explore the deployment of Zetta.

1. The Zetta server runs on a hardware hub, which can be Raspberry Pi, Intel Edison or BeagleBone Black.
2. The hub is connected to devices, and they communicate via HTTP to the specific protocols used in the deployment.
3. Another similar server runs in the cloud, which has the same Node.js packages that are available on the Zetta server in the hub. Both the servers are connected.
4. Zetta provides an API at the cloud endpoint so that consumers can use it.

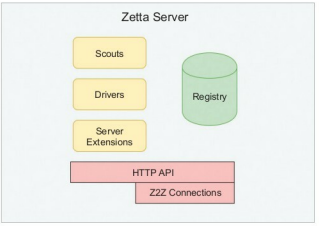

Figure 1: Zetta architecture

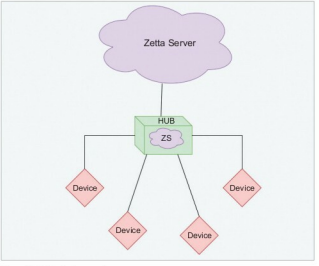

Figure 2: Zetta deployment

Hardware requirements: Zetta runs with approximately six connected devices per hub, which is the ideal scenario suggested by it. Hardware requirements are dependent upon the number of devices, the load of each device and the type of data flowing between them. The ideal minimum requirement is a 500MHz CPU, 500MB RAM and storage of 1GB-2GB. Zetta requires 500MB to be able to run. It supports all common operating systems with 32-bit and 64-bit versions.

Zetta installation and demo project

Now let's take a look at installing Zetta and a 'Hello world' version of the Zetta sample project. Before starting Zetta, here's a brief introduction to Node.js.